OinbuPebas

Plataforma acadêmica de mobilidade urbana para Parauapebas

Integrantes: [preencher]

Matrículas: [preencher]

Turma/disciplina: [preencher]

Instituição: [preencher]

Data: 28 de outubro de 2026

Sumário

1. Contexto e problema

2. Objetivos, escopo e atores

3. Requisitos e casos de uso

4. Dados e proveniência

5. Arquitetura e comunicação

6. Consistência, falhas e segurança

7. Implantação e validação

8. Limitações e conclusão

1. Contexto e problema

O acesso a informações de transporte coletivo exige mais do que desenhar uma linha entre dois pontos. Uma decisão útil combina acesso à parada, horário de chegada do passageiro, partidas ainda possíveis, sentido e ordem, conexão e caminhada final. OinbuPebas demonstra esse raciocínio para Parauapebas sem se apresentar como serviço oficial.

2. Objetivos, escopo e atores

O objetivo é permitir consulta pública de catálogo, planejamento e veículos simulados, com conta opcional para favoritos. **USER** administra seus dados; **ADMIN** mantém catálogo, tarefas e simulação. Venda, pagamento, reserva, telemetria real e controle operacional estão excluídos. Se o sistema falhar, o transporte real continua normalmente.

3. Requisitos e casos de uso

O fluxo de aceite parte do bairro Tropical e chega ao Partage Shopping, com uma baldeação demonstrativa na Parada Amazônica. A busca recebe data, saída, até duas baldeações e caminhada. Resultados explicam duração, espera, caminhada, veículos e proveniência. Casos detalhados estão em **casos-de-uso.md** e regras numeradas em **regras-de-negocio.md**.

4. Dados e proveniência

Publicações municipais de 2025 sustentam nomes de linhas e integração na Parada Amazônica, mas não uma grade completa atual. Planos antigos apenas contextualizam Tropical/IB003. Assim, a rede usa nomes confirmados com horários, geometrias e conexões explicitamente sintéticos. São 9 linhas, 44 paradas e até 12 veículos derivados de viagens simuladas; cada entidade expõe proveniência. O Partage usa endereço do site oficial e coordenada OSM/ODbL. Detalhes e URLs constam em **dados-e-fontes.md**.

5. Arquitetura e comunicação

Foi adotado monólito modular Spring Boot com seis módulos. O navegador usa REST/JSON para comandos/consultas e SSE para posições; polling é fallback. Chamadas Java explícitas atendem operações internas imediatas. RabbitMQ recebe tarefas e eventos persistentes por outbox; não participa de consultas comuns. PostgreSQL/PostGIS é único e mantém propriedade lógica de tabelas. Caddy faz TLS/roteamento, não API Gateway. Não existe Service Mesh nem microsserviço disfarçado.

O planejador demonstrativo percorre partidas ordenadas, elimina as perdidas, aplica conexão mínima e soma caminhada/espera/viagem. A simulação interpola uma polilinha compartilhada com referência temporal do backend. O modelo SQL separa linha, padrão/sentido, parada, sequência, calendário, viagem e stop time.

6. Consistência, falhas e segurança

Outbox transacional preserva eventos quando RabbitMQ falha; confirmação correlacionada só marca publicação após ACK. A chave de evento processado permite efeito idempotente; retry, DLQ e reprocessamento auditado cobrem falhas persistentes. Edição usa versão otimista. A entrega é pelo menos uma vez, sem promessa de exactly-once.

Sessões são HttpOnly/SameSite, Secure em produção, CSRF é habilitado e senhas usam BCrypt. Autorização ocorre no backend e favoritos são filtrados pelo proprietário. Upload tem limite, erros usam Problem Details e API/PWA evitam cache indiscriminado de resposta privada. Localização só é solicitada após ação e não é persistida continuamente.

Falhas de backend, timeout, mapa e SSE possuem estado explícito. Banco indisponível derruba readiness; nenhum fallback inventa dado persistido. O cenário local do frontend é marcado como demonstração e serve apenas para comunicar indisponibilidade.

7. Implantação e validação

Dockerfiles multi-stage, Compose, healthchecks, volumes, limites, logs rotativos e Caddy TLS foram preparados. PostgreSQL/RabbitMQ não são publicados na produção. Backups usam **pg_dump -Fc**; rollback de app preserva cautela com migrações.

Foram executados testes Maven, Vitest e build Vite. O host de desenvolvimento não possuía Docker, portanto os testes Testcontainers locais e Playwright não foram executados. Após autorização explícita, o commit **2f54e11** foi publicado na VPS e validado externamente em : migrations e seed, catálogo, planejamento Tropical → Partage, SSE, cadastro/login/logout, favoritos persistidos e autorização administrativa sem sessão. O DNS usa um CNAME para um túnel Cloudflare dedicado ao projeto; banco, broker, Mailpit e Actuator não são expostos publicamente.

8. Limitações e conclusão

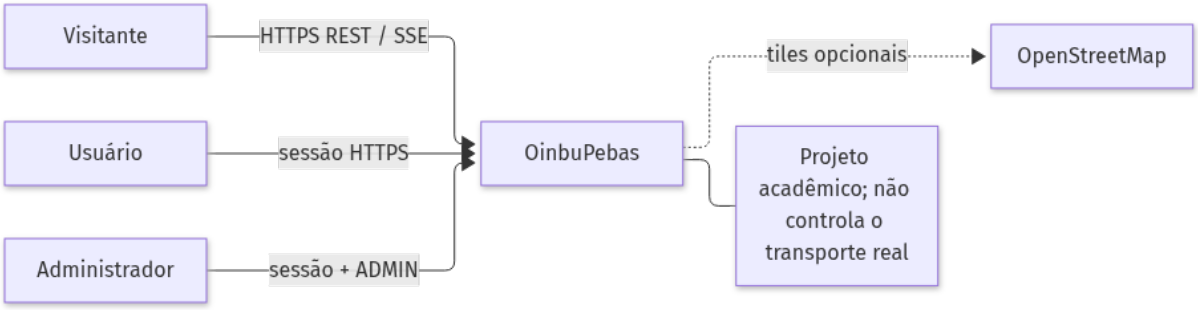
O sistema entrega fundação executável, interface responsiva, planejamento temporal genérico sobre catálogo persistido, sessão, favoritos, simulação derivada de viagens, recuperação por token, administração, importação CSV, outbox e consumidor idempotente. A suíte de integração e E2E real permanece não validada porque o host não possui Docker nem navegador Chromium disponível; não se afirma definição de pronto integral até esses bloqueios externos serem removidos.

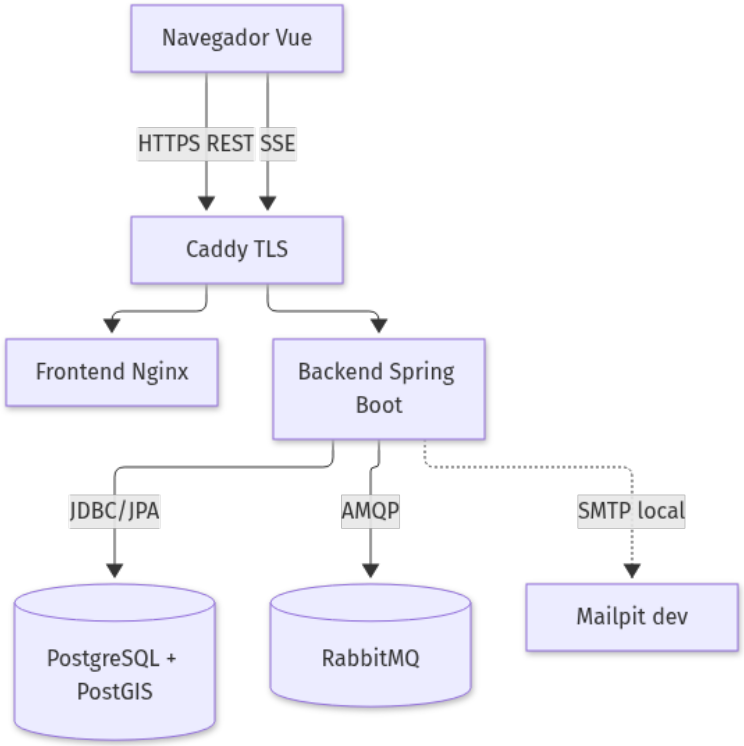
A contribuição acadêmica principal é tornar explícita a diferença entre proximidade espacial e viabilidade temporal, sem ocultar proveniência ou criar autoridade operacional inexistente.

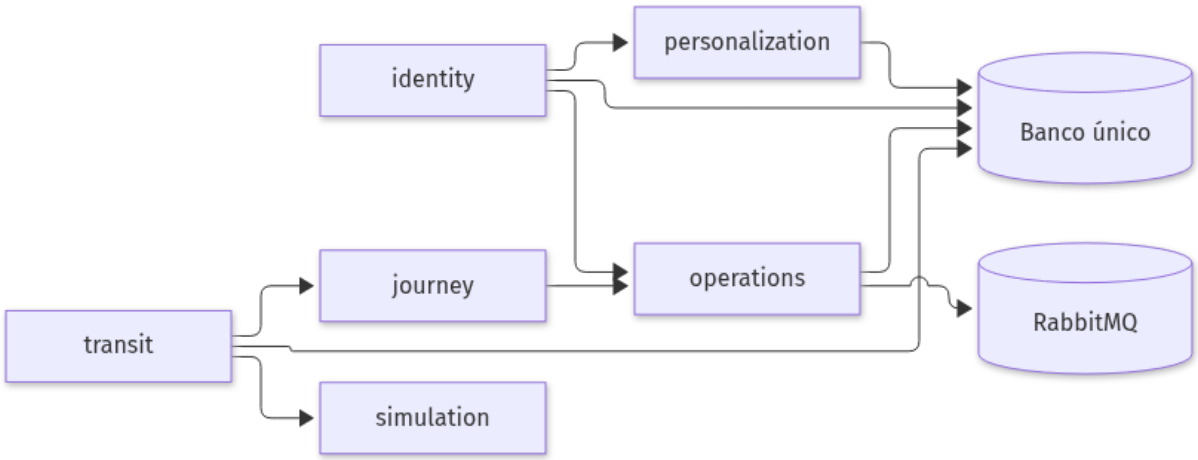
Fontes consultadas

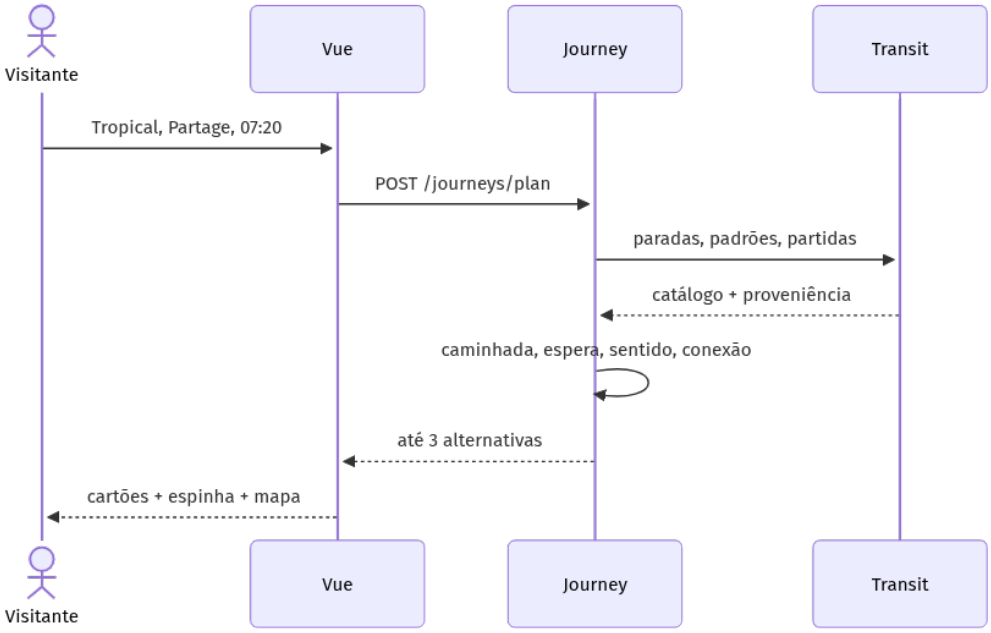
Consulte a tabela completa, datas, campos sustentados e limitações em **dados-e-fontes.md**.

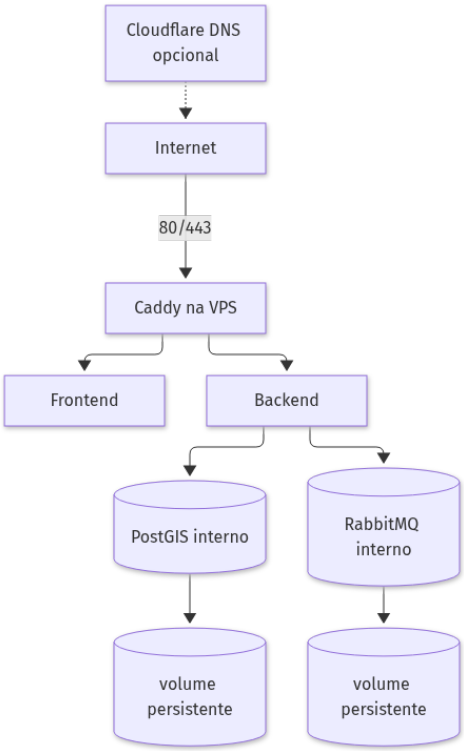
Diagramas selecionados








Visitante
Vue
Journey
Transit
Tropical, Partage, 07:20
POST /journeys/plan
paradas, padrões, partidas
catálogo + proveniência
caminhada, espera, sentido, conexão
até 3 alternativas
cartões + espinha + mapa
Visitante
Vue
Journey
Transit


Cloudflare DNS opcional
Internet
80/443
Caddy na VPS
Frontend
Backend
PostGIS interno
RabbitMQ interno
volume persistente
volume persistente